

Dungeon Serpent

Software Requirements Specification

Ray Buenviaje | WSU CptS 322 | 2026

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the requirements for **Dungeon Serpent**, a 2D fantasy-themed Snake game developed in C# using the MonoGame 3.8.1 DesktopGL framework. The document defines the system's functional behavior, non-functional constraints, and use cases for the developer and course instructor.

Dungeon Serpent is a single-player desktop game where the player steers a growing serpent across a dungeon tile grid, collecting gems to score points and grow in length. The game ends on wall or self-collision. It supports a title screen, active gameplay with a live HUD, pause/resume, game-over, and local high-score saving.

1.2 Reference Documents

- Microsoft .NET 8 Documentation — <https://learn.microsoft.com/en-us/dotnet/>
- MonoGame Framework 3.8.1 Documentation — <https://docs.monogame.net/>
- MonoGame NuGet Package MonoGame.Framework.DesktopGL v3.8.1.303 — <https://www.nuget.org/packages/MonoGame.Framework.DesktopGL>
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications
- WSU CptS 322 Assignment 2 Rubric — provided by course instructor
- C# Language Reference — <https://learn.microsoft.com/en-us/dotnet/csharp/>

1.3 Abbreviations and Acronyms

- **SRS** — Software Requirements Specification
- **UI** — User Interface
- **HUD** — Heads-Up Display (in-game bar showing score, level, gem info)
- **FPS** — Frames Per Second
- **DGL** — DesktopGL (MonoGame cross-platform desktop rendering target)
- **NuGet** — Package manager for .NET dependencies
- **UC** — Use Case

- **SDK** — Software Development Kit
- **UML** — Unified Modeling Language
- **WASD** — Keyboard movement keys (W=Up, A=Left, S=Down, D=Right)

2 General Description

2.1 Product Description

Dungeon Serpent is a 2D single-player desktop game built with C# and MonoGame 3.8.1 DesktopGL. The player controls a serpent moving across a 25×22 tile dungeon grid (28 px/tile). The serpent grows each time it consumes a gem, and the game ends on wall or self-collision. Four gem types exist—Ruby, Sapphire, Amethyst, and Topaz—each with escalating point values. The dungeon features pre-baked stone tiles, flickering torches at six fixed positions, and animated gem glow effects.

The game lifecycle includes a title screen, active gameplay with a persistent HUD, pause overlay, and game-over overlay. The HUD shows score, level, gem type, serpent length, and controls. Difficulty scales with level: every 5 gems collected increases speed. High scores are saved to a local plaintext file between sessions. No network connectivity is required.

2.2 Operating Environment

- **Operating System:** Linux (Ubuntu 20.04+), macOS 12+, or Windows 10/11 (64-bit)
- **Runtime:** .NET 8 SDK — <https://dotnet.microsoft.com/download>
- **Graphics:** MonoGame DesktopGL; compatible with integrated or dedicated GPUs
- **Hardware:** Standard PC/laptop with keyboard; minimum 512MB RAM, 200 MB disk
- **File Structure:** `highscores.txt` written to the working directory; no database used
- **Network:** Not required; fully offline
- **Display:** Minimum 800×700 px; default window is 700×696 px

2.3 General Constraints, Assumptions, Dependencies, Guidelines

- Single-user only; no concurrent user or network support
- Keyboard-only input; mouse is not used
- .NET 8 SDK must be installed before building or running
- MonoGame 3.8.1 must be resolvable via NuGet at build time
- Three font `.spritefont` assets (`GameFontLg`, `GameFontMd`, `GameFontSm`) must be in `Content/`

- Courier New font must be installed on the OS for SpriteFont rendering
- `highscores.txt` is created automatically on first run if absent
- No dedicated GPU assumed; integrated graphics are sufficient
- Controller and touch input are not supported in version 1.0

2.4 Design and Implementation Constraints

- SHALL be implemented in C# targeting .NET 8
- SHALL use MonoGame Framework **v3.8.1.303** (`MonoGame.Framework.DesktopGL`)
- SHALL use `MonoGame.Content.Builder.Task v3.8.1.303` for font pipeline compilation
- SHALL follow object-oriented principles; core logic in `DungeonSerpentGame` extending `Game`
- SHALL render all graphics programmatically via `SpriteBatch`; no external image assets
- SHALL target DesktopGL (cross-platform: Windows, macOS, Linux)
- SHALL NOT use third-party libraries beyond MonoGame and the .NET 8 standard library
- High score persistence SHALL use `System.IO` plaintext file only
- Window resolution SHALL be fixed at 700×696 px in version 1.0

3 Use Cases

3.1 Use Case Diagram

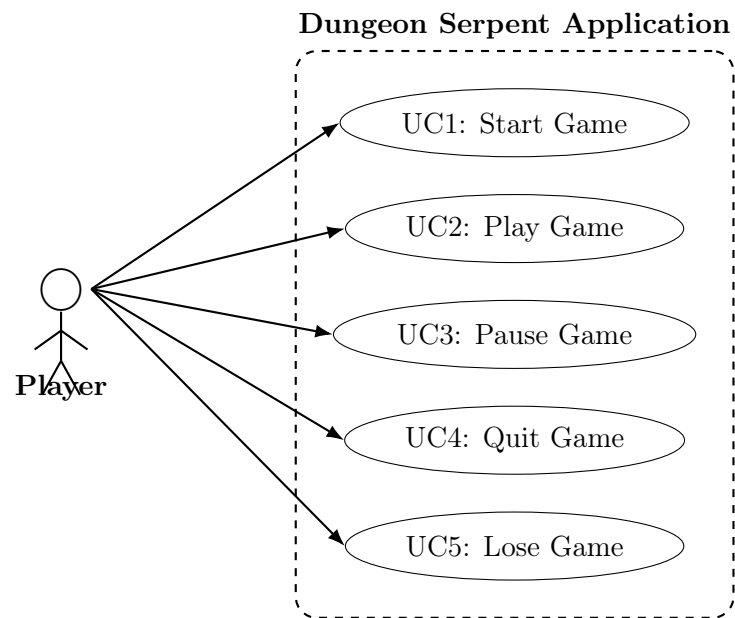


Figure 1: Dungeon Serpent UML Use Case Diagram

3.2 Use Cases

3.2.1 Use Case 1 — Start Game

Title	Start Game
Description	Player launches the app, views the title screen, and starts a new game by pressing Enter.
Actors	Player; Keyboard (Enter); <code>DungeonSerpentGame</code> title screen state
Preconditions	Application launched, game window open, title screen displayed with blinking prompt.
Steps	1. System initializes the 700×696 px window and loads fonts and textures. 2. System displays the animated title screen (floor tiles, torches, pulsing title). 3. Player reads on-screen instructions (gem types, controls, level-up rule). 4. Player presses Enter. 5. System sets <code>_titleScreen = false</code> and calls <code>ResetGame()</code>. 6. Serpent initialized at grid center, direction = right, score = 0, level = 1; first gem spawned. 7. Main game loop begins; dungeon gameplay screen rendered.
Post Conditions	Game in active play state. Serpent moving right at center. HUD shows Score: 0, Level: 1.
Alternative Flow	Pressing Escape on the title screen calls <code>Exit()</code> and terminates the app.

3.2.2 Use Case 2 — Play Game

Title	Play Game
Description	Player steers the serpent to collect gems, score points, and advance levels until collision or pause.
Actors	Player; Keyboard (WASD / Arrow Keys); game update loop
Preconditions	<code>_titleScreen = false</code> , <code>_gameOver = false</code> , <code>_paused = false</code> ; gem present on grid.
Steps	1. Each frame, system reads directional input and updates <code>_nextDir</code> if valid (non-reversing). 2. On each logic step (gated by <code>_stepTimer</code>), serpent head advances one tile. 3. System checks wall collision; if hit, sets <code>_gameOver = true</code>. 4. System checks self-collision; if hit, sets <code>_gameOver = true</code>. 5. If head matches gem position: award pts (<code>gem.Points × level</code>), grow serpent, spawn new gem. 6. Every 5 gems, increment level, decrease <code>_stepInterval</code>, show level-up message. 7. System renders dungeon, serpent, gem, particles, messages, and HUD at 60 FPS.
Post Conditions	Serpent advanced one tile. On gem: score up, serpent longer. On collision: Game Over.
Alternative Flow	P pauses (UC3). Escape exits. Reverse-direction input is ignored.

3.2.3 Use Case 3 — Pause Game

Title	Pause Game
Description	Player pauses active gameplay, freezing all movement and timers, and views a pause overlay.
Actors	Player; Keyboard (P key); <code>_paused</code> flag
Preconditions	Game is in active play state; <code>_gameOver = false</code> , <code>_titleScreen = false</code> .
Steps	1. Player presses P during active gameplay. 2. System detects edge-triggered keydown for <code>Keys.P</code>; sets <code>_paused = true</code>. 3. System halts <code>_stepTimer</code> and stops all game logic updates. 4. System renders semi-transparent overlay: “[PAUSED]” and “Press P to continue”. 5. Player presses P again. 6. System sets <code>_paused = false</code>; game loop resumes from exact paused state.
Post Conditions	Game resumes with no state changed (score, level, serpent, gem position all unchanged).
Alternative Flow	Escape while paused exits the app. Pause unavailable on title or game-over screens.

3.2.4 Use Case 4 — Quit Game

Title	Quit Game
Description	Player terminates the application cleanly from any state.
Actors	Player; Keyboard (Escape) or OS window close button
Preconditions	Application is running in any state (title, play, paused, or game over).
Steps	1. Player presses Escape or clicks the OS window close button. 2. System calls <code>Game.Exit()</code>, signaling <code>MonoGame</code> to end the loop. 3. <code>MonoGame</code> disposes all managed resources (<code>SpriteBatch</code>, <code>Texture2D</code>, <code>SpriteFont</code>). 4. Application process terminates; window closes.
Post Conditions	Application terminated. All in-memory state released. No confirmation dialog shown.
Alternative Flow	On game-over screen, pressing R restarts instead of quitting.

3.2.5 Use Case 5 — Lose Game

Title	Lose Game
Description	Serpent collides with a wall or itself, triggering game over. Player views final score and may restart or quit.
Actors	Player (passive); Keyboard (R to restart, Esc to quit)
Preconditions	Game in active play state; serpent's next head position equals a wall tile or body segment.
Steps	1. <code>StepSnake()</code> calculates next head position from current direction. 2. If next position is at a boundary tile, system sets <code>_gameOver = true</code>. 3. If next position matches any body segment, system sets <code>_gameOver = true</code>. 4. System halts game logic and renders game-over overlay: "SERPENT SLAIN", final score, "R to restart". 5a. Player presses R: <code>ResetGame()</code> called; game restarts (score 0, level 1). 5b. Player presses Escape: application exits (UC4).
Post Conditions	If R: game restarted. If Escape: app terminated. Directional keys on game-over screen are ignored.
Alternative Flow	OS window close exits normally from the game-over screen.

4 Requirements

4.1 Functional Requirements

4.1.1 Main Menu / Title Screen

- SHALL display an animated title screen on launch before any gameplay begins.
- SHALL render the title "DUNGEON SERPENT" with a pulsing gold color animation.
- SHALL display a blinking "Press ENTER to begin..." prompt (alternates every 20 frames).
- SHALL show tip lines for gem types, controls, and the level-up rule.
- SHALL render dungeon floor tiles and animated torches behind the title overlay.
- SHALL transition to active gameplay when Enter is pressed.
- SHALL exit the application when Escape is pressed on the title screen.
- SHALL display the HUD bar at the bottom of the title screen.
- SHOULD display the game version and author name on the title screen.

4.1.2 Game Play and Interaction

- SHALL maintain a 25×22 tile grid at 28 px/tile as the playfield.
- SHALL initialize the serpent with 3 segments at the grid center, moving right.
- SHALL accept directional input from WASD and Arrow Keys.
- SHALL prevent the serpent from reversing direction.
- SHALL advance the serpent one tile per logic step via a step timer.
- SHALL detect wall collision and transition to Game Over.
- SHALL detect self-collision and transition to Game Over.
- SHALL grow the serpent by one segment per gem consumed.
- SHALL spawn a new gem at a random valid (unoccupied) position after each gem is collected.
- SHALL render the serpent head with directional eyes and body segments with fading scale detail.
- SHALL render gems with bobbing animation and a glow effect.
- SHALL render flickering torch animations at six fixed positions.
- SHALL display a pause overlay when P is pressed; resume when P is pressed again.
- SHALL spawn particle effects and a floating “+N pts” message on gem collection.
- SHALL render at 60 FPS; game logic runs on an independent step interval timer.

4.1.3 Scoring, Save, and Load

- SHALL calculate gem points as: base points $\times$ current level (e.g., Ruby at level 3 = $10 \times 3 = 30$).
- SHALL display the current score in the HUD throughout gameplay.
- SHALL increment the level by 1 every 5 gems consumed.
- SHALL display a “LEVEL N!” floating message on level-up.
- SHALL decrease the step interval on each level-up (minimum 0.04 seconds).
- SHALL display current level, gem type, point value, and serpent length in the HUD.
- SHALL save the final score to `highscores.txt` on game over.
- SHALL load and display top scores from `highscores.txt` on the title or game-over screen.
- SHALL handle a missing `highscores.txt` gracefully (create on first run).
- SHOULD display the all-time high score in the HUD for reference.

4.2 Non-Functional Requirements

4.2.1 Performance Requirements

- SHALL render consistently at 60 FPS on hardware meeting minimum specifications.
- SHALL produce no visible screen flickering during animations or particle effects.
- SHALL complete each logic step in under 16 ms (one frame at 60 FPS).
- SHALL pre-bake floor tile colors at startup (seeded RNG, seed 42) to avoid per-frame recalculation.
- SHALL cap speed at a minimum step interval of 0.04 seconds (25 steps/sec).
- SHALL use a single 1×1 white `Texture2D` for all primitive drawing to minimize GPU state changes.
- SHOULD remove expired particles and messages each frame to prevent unbounded memory growth.

4.2.2 Usability

- SHALL display controls (WASD/Arrows, P=Pause, R=Restart) in the HUD at all times.
- SHALL use distinct colors per game element: gold=score, green=serpent, orange=torches, unique color per gem.
- SHALL provide immediate visual feedback on gem collection via particles and floating score text.
- SHALL display clear game-over and pause overlays with explicit next-action instructions.
- SHALL show a title screen with instructions before the first play session.
- SHALL display current gem name and point value in the HUD at all times.
- SHOULD use a monospace font (Courier New) for HUD and overlay text.